BÖLÜM

5

Çoklu Form Uygulamaları



BU BÖLÜMÜN SONUNDA

- Projeye yeni form ekleyip formun başlangıç konumunu ayarlayabilecek,
- Bir formdan başka bir formu nesne oluşturarak açabilecek,
- Show ile ShowDialog arasındaki farkı kod akışı üzerinden açıklayabilecek,
- Hide, Close ve Application.Exit komutlarının etkisini ayırt edebilecek,
- Gizlenen ana form yüzünden kapanmayan program sorununu çözebilecek,
- Formlar arasında kurucu metot ile veri aktarabileceksiniz.

KampüsCepte · öğrenciler için ücretsiz ders notları

Gerçek uygulamalar tek pencereden oluşmaz: önce bir giriş ekranı, ardından ana menü, oradan da ayar ya da rapor pencereleri açılır. Bu bölümde bir formdan başka bir formu açmayı, gizlemeyi ve kapatmayı öğreniyoruz.

5.1 Projeye Yeni Form Ekleme

1. **Project** menüsünden **Add Windows Form...** alt menüsüne tıklanır.
2. Açılan *Add New Item* penceresinde **Windows Form** seçilir. **Name** alanında varsayılan ad `Form2.cs`'dir. İstenirse değiştirilir.
3. **Add** butonuna basılır. Solution Explorer'da `Form2.cs` görünür ve yeni formun tasarım penceresi açılır.

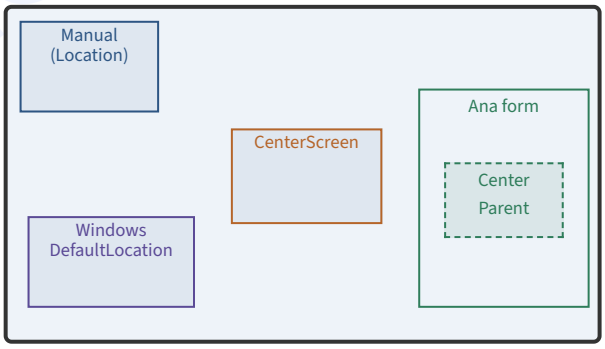
> **NOT**
> Yeni Visual Studio sürümlerinde aynı komut **Project → Add Form (Windows Forms)...** ya da Solution Explorer'da projeye sağ tıklayıp **Add → Form (Windows Forms)...** şeklinde bulunur.

5.2 Formun Başlangıç Konumu: StartPosition

Hazırlanan formun ekrana ilk geldiği konum ayarlanabilir. Bunun için form seçildikten sonra Properties penceresindeki **StartPosition** özelliğinden ilgili değer seçilir. Kod ile de ayarlanabilir:

```
this.StartPosition = FormStartPosition.CenterScreen;
```

Değer	Form ekrana nasıl gelir?
`Manual`	Location (konum) özelliğinde belirtilen koordinatlarda
`CenterScreen`	Ekranın tam ortasında
`WindowsDefaultLocation`	Windows'un belirlediği konumda (yeni projelerde varsayılan)
`WindowsDefaultBounds`	Konumu ve boyutu Windows'un belirlediği şekilde
`CenterParent`	Kendisini açan ana formun ortasında. En güvenilir biçimde ShowDialog ile açılan formlarda çalışır.



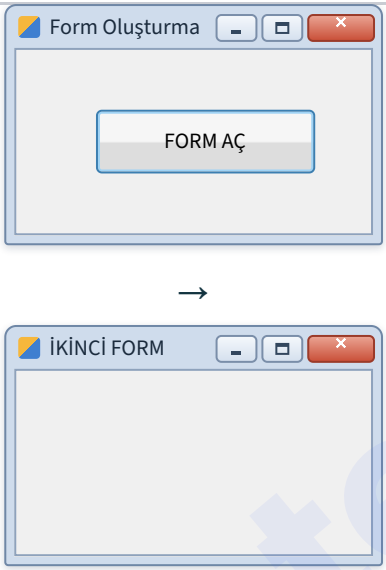
Şekil 5.1 *StartPosition değerleri*

5.3 Form İçinden Yeni Form Açma

Form1'e **FORM AÇ** butonu eklenir ve butona çift tıklanarak Click metoduna aşağıdaki kodlar yazılır:

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button1_Click(object sender,
EventArgs e)
{
    Form2 yeni = new Form2();   // Form2
sınıfından yeni bir nesne oluşturulur
    yeni.Show();                // Form2
ekranda gösterilir
}
```



NESNE TABANLI BAKIŞ

Projeye eklenen `Form2` bir **sınıftır**. Tasarım penceresinde gördüğümüz yalnızca kalıptır. `new Form2()` bu kalıptan bellekte gerçek bir pencere **nesnesi** üretir ve bu nesneye `yeni` adını verir. Nesne oluşturmak, onu ekranda göstermek demek değildir. `yeni.Show()` metodu çağrılana kadar pencere görünmez. Butona her tıklandığında `new` yeniden çalışır ve her seferinde **yeni bir Form2 nesnesi**, yani yeni bir pencere oluşur.

5.4 Show ve ShowDialog

Form iki farklı yöntemle gösterilebilir:

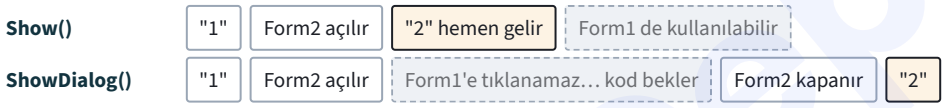


ShowDialog: önceki işlem bitmeden sıradaki başlayamaz

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button1_Click(object sender, EventArgs e)
{
    Form2 yeni = new Form2();
    yeni.ShowDialog();
    // ShowDialog: açılan pencere kapatılmadan diğer pencerelere ERİŞİLEMEZ
    // Show      : açılan pencere kapatılmadan da diğer pencerelere ERİŞİLİR
}
```

	Show()	ShowDialog()
Pencere türü	Modsuz (modeless)	Kalıcı / modal
Açan forma erişim	Açılan pencere kapanmadan da diğer pencerelere erişilir	Açılan pencere kapatılmadan diğer pencerelere **erişilemez**
Sonraki kod satırları	Hemen çalışır	Açılan pencere kapanana kadar **bekler**
Tipik kullanım	Ana menüden bağımsız pencereler	Ayarlar, onay, bilgi girişi pencereleri



Şekil 5.2 `MessageBox.Show("1"); f.Show…(); MessageBox.Show("2");` kodunun iki yöntemdeki akışı

5.5 Ana Formu Gizleme ve Formu Kapatma

Yeni form açıldığında ana formun ekranda kalması istenmiyorsa ana form gizlenir. `this` kodun içinde bulunduğu formu (burada Form1'i) ifade eder:

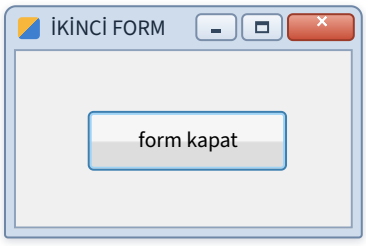
FORM1: FORM2'Yİ AÇIP KENDİNİ GİZLER

```
private void button1_Click(object sender, EventArgs e)
{
    this.Hide();              // şu anki formu (Form1) gizler
    Form2 yeni = new Form2();   // yeni formun nesnesi oluşturulur
    yeni.Show();
}
```

Form2'ye bir **form kapat** butonu konursa, Click metoduna yazılan `this.Close()` yalnızca kullanıldığı formu kapatır:

FORM2: KENDİNİ KAPATIR

```
private void button1_Click(object sender, EventArgs e)
// Form2 üzerindeki "form kapat"
{
    this.Close();   // yalnızca içinde bulunulan formu
(Form2) kapatır
}
```



KampüsCepte · öğrenciler için ücretsiz ders notları

Komut	Ne yapar?	Uygulamaya etkisi
`this.Hide()`	Formu gizler (`Visible = false`). Nesne bellekte durur, daha sonra `Show()` ile yeniden gösterilebilir.	Uygulama çalışmaya devam eder
`this.Close()`	Formu kapatır.	Kapatılan form **ana form** (Program.cs'de `Application.Run` 'a verilen) ise tüm uygulama kapanır. Değilse yalnızca o form kapanır.
`Application.Exit()`	Uygulamanın tüm formlarını kapatır.	Program sonlanır

DİKKAT

Kapanmayan program sorunu. Form1 `Hide()` ile gizlenip Form2 açıldıktan sonra kullanıcı Form2'yi sağ üstteki X ile kapatırsa ekranda hiç pencere kalmaz, ancak **program çalışmaya devam eder**. Uygulamanın ana formu Form1'dir ve Form1 kapatılmamış, yalnızca gizlenmiştir. Program Görev Yöneticisi'nde açık kalır, Visual Studio da hâlâ "çalışıyor" durumunda bekler. Çözüm, Form2 kapandığında ne olacağını açıkça belirtmektir:

```
private void button1_Click(object sender, EventArgs e)
{
    Form2 yeni = new Form2();
    yeni.FormClosed += Yeni_FormClosed;   // Form2 kapanınca çalışacak metodu bağla
    yeni.Show();
    this.Hide();
}

private void Yeni_FormClosed(object sender, FormClosedEventArgs e)
{
    this.Show();              // Form2 kapanınca ana form yeniden görünür
    // Uygulamanın tamamen kapanması istenirse: Application.Exit();
}
```

Buradaki `yeni.FormClosed += Yeni_FormClosed;` satırı, 1. bölümde Designer dosyasında gördüğümüz olay bağlama işleminin aynısıdır. Bu kez bağlantıyı biz kod ile kurarız. Form2 hangi yolla kapanırsa kapansın (X, Close, Alt+F4) FormClosed olayı tetiklenir ve Form1 yeniden görünür.

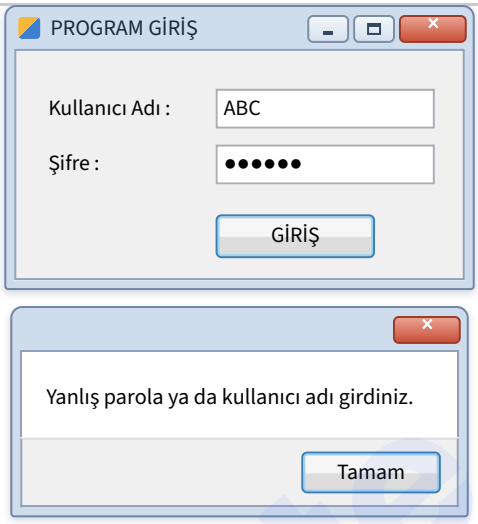
5.6 Uygulama: Kullanıcı Giriş Formu

Form1 üzerine kullanıcı adı ve şifre kutuları ile GİRİŞ butonu eklenir. Kullanıcı bilgileri doğruysa Form2 açılsın, değilse hata uyarı mesajı verilsin.

KampüsCepte · öğrenciler için ücretsiz ders notları

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs
e)
{
    string kadi = "a", ksifre = "a";
    string ad, sifre;
    ad = textBox1.Text;
    sifre = textBox2.Text;
    if (ad == kadi && sifre == ksifre)
    {
        Form2 ali = new Form2();
        ali.Show();
        this.Hide();
    }
    else
    {
        MessageBox.Show("Yanlış parola ya da
kullanıcı adı girdiniz.");
    }
}
```



Uygulamayı geliştirmek için şu noktalara dikkat edilmelidir:

- Şifre kutusunun `PasswordChar` özelliği `*` yapılmalıdır. Şifre ekranda açık görünmemelidir.
- Giriş başarılı olunca Form1 gizlendiği için, Form2 kapatıldığında uygulamanın da kapanması sağlanmalıdır (Bölüm 5.5).
- Gerçek uygulamalarda kullanıcı adı ve şifre koda yazılmaz, veri tabanında (şifre özetlenerek) saklanır. Buradaki sabit değerler yalnızca öğretim amaçlıdır.
- Sınırsız deneme hakkı tanımak güvenlik açığıdır. Deneme sayısını sınırlamak bu bölümün Kendini Test Et kısmında Soru 5.4 olarak size bırakılmıştır.

5.7 Nesne Tabanlı Bakış: Formlar Arası Veri Aktarma

Giriş yapan kullanıcının adını Form2'de göstermek istediğimizi düşünelim. Form2, Form1'deki `textBox1`'e doğrudan erişemez, çünkü o kontrol Form1 nesnesinin **private** (özel) bir parçasıdır. Doğru yol, veriyi Form2 nesnesi oluşturulurken ona **göndermektir**. Bunun için Form2'ye **parametreli bir kurucu metot** yazılır:

FORM2.CS

```
using System;
using System.Windows.Forms;

namespace Giris
{
    public partial class Form2 : Form
    {
        public Form2(string kullaniciAdi)       // parametreli kurucu metot
        {
            InitializeComponent();
```


KampüsCepte · öğrenciler için ücretsiz ders notları

```
            this.Text = "Hoş geldin, " + kullaniciAdi;
        }
    }
}
```

FORM1.CS

```
using System;
using System.Windows.Forms;

namespace Giris
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Form2 anaSayfa = new Form2(textBox1.Text);   // veri kurucuya gönderilir
            anaSayfa.Show();
        }
    }
}
```

`new Form2(textBox1.Text)` ifadesi, Form2 nesnesi oluşturulurken kurucu metodu çalıştırır ve kutudaki metni `kullaniciAdi` parametresine aktarır. Kurucu metot bu değeri formun başlığına yazar. Bu yöntemde her form kendi kontrollerinden sorumludur ve veri formlar arasında açıkça tanımlanmış bir kapıdan, kurucu metotdan geçer. Nesne tabanlı programlamada buna **kapsülleme** denir.

DİKKAT

Form2'ye parametreli kurucu yazıldıktan sonra parametresiz `new Form2()` çağrısı derleme hatası verir. Projede iki kullanım birden gerekiyorsa Form2'de her iki kurucu da yazılır.

ÖZET

- Yeni form **Project → Add Windows Form** ile eklenir. Başlangıç konumu **StartPosition** ile belirlenir.
- Form açmak iki adımdır: `Form2 f = new Form2();` nesneyi oluşturur, `f.Show();` ya da `f.ShowDialog();` onu gösterir. Her `new` yeni bir pencere demektir.
- **Show** açan forma erişime izin verir ve kod beklemeden devam eder. **ShowDialog** ise açılan pencere kapanana kadar hem kullanıcıyı hem kodu bekletir.
- `Hide` gizler, `Close` kapatır, `Application.Exit` tüm uygulamayı sonlandırır. Ana form kapatılırsa program biter.
- Ana form gizlendiyse ikinci formun **FormClosed** olayında ana form yeniden gösterilmeli ya da uygulama kapatılmalıdır.
- Formlar arası veri, parametreli kurucu metot ile aktarılır.

KampüsCepte · öğrenciler için ücretsiz ders notları